

3-Day Task-Based Training Plan

C#, .NET, ASP.NET Core Web API with EF Core InMemory Database

System: Trainee Management API

Training Objective

- Build a small REST API gradually using C#, ASP.NET Core Web API, JSON, DTOs, validation, service layer, and EF Core InMemory Database.
- Keep Week 1 focused on API fundamentals and avoid real database setup complexity.
- Move real database integration, EF Core Code First migrations, JWT, React integration, and advanced security to the next phase.

Important Week 1 Scope Decision

- Use EF Core InMemory Database only for Week 1.
- Do not configure SQL Server, MySQL, connection strings, or migrations during the first 3 days.
- Use the same project in Phase 2 to move to EF Core Code First with a real database.

Final Deliverable After 3 Days

Each trainee should submit a working ASP.NET Core Web API project named:

TraineeManagement.Api

Area	Expected Output
Project structure	Controllers, Models, DTOs, Services, Data
Storage	EF Core InMemory Database
Testing	Swagger screenshots or Postman collection
Documentation	README with API list, sample JSON, setup steps, limitations, and next phase scope

Working API endpoints:

- GET /api/health
- GET /api/trainees
- GET /api/trainees/{id}
- POST /api/trainees
- PUT /api/trainees/{id}
- DELETE /api/trainees/{id}
- GET /api/trainees?search=amit

System Entity: Trainee

Field	Type	Required	Notes
Id	int	Auto-generated	Unique identifier

FirstName	string	Yes	Maximum 50 characters
LastName	string	Yes	Maximum 50 characters
Email	string	Yes	Valid email format
TechStack	string	Yes	Example: HTML, CSS, JavaScript
Status	string	Yes	Active, Inactive, Completed
CreatedDate	DateTime	Auto-generated	Set during create
UpdatedDate	DateTime	Auto-generated	Set during update

Day 1: C# Basics and First ASP.NET Core API

Day 1 Goal

- What is C#?
- What is .NET?
- What is ASP.NET Core Web API?
- What is a controller?
- What is a route?
- What is JSON?
- What are HTTP methods?
- How to test APIs using Swagger.

Task	Deliverable	Concepts Covered	Official Reference Links
1.1 Create Web API project	Create TraineeManagement.Api. Run the project and open Swagger/OpenAPI UI.	ASP.NET Core project structure, Program.cs, controllers, Swagger/OpenAPI	Create controller-based Web API ASP.NET Core Overview OpenAPI in ASP.NET Core
1.2 Create Health Check API	Create GET /api/health returning status, application name, and timestamp.	Controller, routing, GET method, JSON response	Create Web APIs with ASP.NET Core Routing to controller actions
1.3 Create Trainee model	Create Trainee class with Id, FirstName, LastName, Email, TechStack, Status, CreatedDate, UpdatedDate.	C# classes, properties, object-oriented basics, DateTime	C# Guide C# Classes and objects
1.4 Create temporary in-memory List	Use List<Trainee> to add, read, and find records temporarily.	Collections, List<T>, application memory	C# Collections
1.5 Create GET and POST APIs	Implement GET /api/trainees, GET /api/trainees/{id}, and POST /api/trainees.	HTTP methods, route parameters, request body, response body, model binding, JSON	Create controller-based Web API Model binding Swagger / OpenAPI help pages

Day 1 Deliverables

- Project runs successfully with Swagger/OpenAPI available.
- Health API is working.
- Trainee model is created.
- GET all, GET by ID, and POST APIs are working with sample JSON.
- Trainee has committed code to Git.

Task 1.1: Create ASP.NET Core Web API Project

Create a new project:

TraineeManagement.Api

Expected output:

- Project runs successfully.
- Swagger opens in browser.
- Trainee understands basic project files:
 - Program.cs
 - appsettings.json
 - Controllers

Task 1.2: Create Health Check API

Create this API:

GET /api/health

Expected response:

```
{
  "status": "running",
  "application": "Trainee Management API",
  "timestamp": "2026-06-03T10:30:00"
}
```

Task 1.3: Create Trainee Model

Create a C# class:

Trainee

Fields:

Id
FirstName
LastName
Email
TechStack
Status
CreatedDate
UpdatedDate

Task 1.4: Create In-Memory Trainee List

Create a temporary in-memory list to store trainee records.

Example:

List<Trainee>

Task 1.5: Create Basic GET and POST APIs

Create these APIs:

```
GET /api/trainees
GET /api/trainees/{id}
POST /api/trainees
```

Sample POST request:

```
{
  "firstName": "Amit",
  "lastName": "Sharma",
  "email": "amit.sharma@training.com",
  "techStack": "HTML, CSS, JavaScript",
  "status": "Active"
}
```

Day 2: DTOs, Validation, Service Layer, PUT and DELETE

Day 2 Goal

- Improve the API design using DTOs, validation, service layer, dependency injection, and proper HTTP status codes.
- Introduce clean code structure without adding real database complexity.

Task	Deliverable	Concepts Covered	Official Reference Links
2.1 Add DTOs	Create CreateTraineeRequest, UpdateTraineeRequest, and TraineeResponse. Use DTOs in controller actions.	API contract design, request/response separation, controlled output	Create Web APIs with ASP.NET Core Model binding
2.2 Add validation	Validate FirstName, LastName, Email, TechStack, and Status. Return clear field-level error messages.	Data annotations, model validation, meaningful errors, accessibility-aware errors	Model validation Data annotations W3C WCAG Error Identification
2.3 Add service layer	Create ITraineeService and TraineeService. Move business logic outside controller.	Interfaces, classes, separation of concerns, dependency injection	Dependency injection in ASP.NET Core Controller dependency injection
2.4 Add PUT API	Implement PUT /api/trainees/{id}. Return 200, 400, or 404 as applicable.	Update operation, route parameter, status codes, validation	Create Web APIs with ASP.NET Core Routing to controller actions Model validation
2.5 Add DELETE API	Implement DELETE /api/trainees/{id}. Return 204 or 404 as applicable.	Delete operation, correct HTTP response, clean API behavior	Create Web APIs with ASP.NET Core Routing to controller actions

Day 2 Deliverables

- DTOs are created and used by APIs.
- Validation is implemented and returns clear field-level errors.
- Service layer is implemented and registered using dependency injection.
- PUT and DELETE APIs are working.
- Correct status codes are used for success, validation error, and not found scenarios.

- Swagger/Postman testing evidence is available.
- Trainee has committed code to Git.

Task 2.1: Add DTOs

Create the following DTOs:

CreateTraineeRequest
UpdateTraineeRequest
TraineeResponse

Purpose:

- CreateTraineeRequest: used when adding a trainee
- UpdateTraineeRequest: used when updating a trainee
- TraineeResponse: used when returning trainee details

Task 2.2: Add Validation

Add validation rules:

Field	Rule
FirstName	Required, max 50 characters
LastName	Required, max 50 characters
Email	Required, valid email format
TechStack	Required
Status	Required and must be valid

Expected validation error example:

```
{
  "errors": {
    "firstName": ["First name is required"],
    "email": ["Valid email is required"]
  }
}
```

Task 2.3: Add Service Layer

Create:

ITraineeService
TraineeService

Suggested structure:

Controllers
Models

DTOs
Services

Controller should only:

- Receive request
- Call service
- Return response

Service should:

- Hold business logic
- Manage trainee data
- Map model to response DTO
- Validate trainee status if needed

Task 2.4: Add PUT API

Create:

PUT /api/trainees/{id}

Purpose:

Update trainee details.

Expected behavior:

Scenario	Response
Valid trainee ID	200 OK
Invalid trainee ID	404 Not Found
Invalid request	400 Bad Request

Task 2.5: Add DELETE API

Create:

DELETE /api/trainees/{id}

Expected behavior:

Scenario	Response
Valid trainee ID	204 No Content
Invalid trainee ID	404 Not Found

Day 3: EF Core InMemory Database, Search, and Readiness Review

Day 3 Goal

- Replace manual List<T> storage with EF Core InMemory Database.
- Introduce DbContext, DbSet, async/await, LINQ filtering, and search query parameters.
- Prepare trainees for Phase 2, where the same project will move to EF Core Code First with a real database.

Task	Deliverable	Concepts Covered	Official Reference Links
3.1 Add EF Core InMemory package	Add Microsoft.EntityFrameworkCore.InMemory and understand why it is used only for training/testing in Week 1.	EF Core provider concept, InMemory database, temporary storage	EF Core overview EF Core InMemory provider
3.2 Create AppDbContext	Create AppDbContext and add DbSet<Trainee> Trainees.	DbContext, DbSet, EF Core data model	EF Core DbContext EF Core overview
3.3 Register InMemory DB in Program.cs	Configure UseInMemoryDatabase("TraineeManagementDb") and inject AppDbContext.	Service registration, dependency injection, application startup configuration	Dependency injection in ASP.NET Core EF Core InMemory provider
3.4 Replace List<T> with EF Core	Update service methods to use AppDbContext with AddAsync, FindAsync, ToListAsync, SaveChangesAsync, and Remove.	EF Core CRUD, async/await, data access through DbContext	EF Core DbContext EF Core async queries
3.5 Add search API	Enhance GET /api/trainees?search=amit to search FirstName, LastName, Email, and TechStack.	Query parameters, LINQ filtering, basic scalable API thinking	C# LINQ Model binding
3.6 Complete quality review	Review JSON, DTOs, validation, status codes, service layer, EF Core usage, async methods, Swagger testing, and README.	API quality, maintainability, readiness for Phase 2	ASP.NET Core Overview OWASP API Security Top 10 W3C WCAG Error Identification

Day 3 Deliverables

- EF Core InMemory setup is completed.
- AppDbContext is created and registered.
- Manual List<T> storage is removed from service logic.
- CRUD APIs work using EF Core InMemory Database.
- Search API is working.
- README is added or updated.
- Swagger/Postman testing evidence is available.
- Trainee has committed code to Git.

Sample JSON Payloads

Sample POST /api/trainees request:

```
{
  "firstName": "Amit",
  "lastName": "Sharma",
  "email": "amit.sharma@training.com",
  "techStack": "HTML, CSS, JavaScript",
}
```

```
"status": "Active"
}
```

Sample validation error response:

```
{
  "errors": {
    "firstName": ["First name is required"],
    "email": ["Valid email is required"]
  }
}
```

README Requirements

- Project Name
- Technology Used
- How to Run
- API List
- Sample Request JSON
- Sample Response JSON
- Known Limitations

Quality Review Checklist

Area	Expected Practice
JSON	Request and response payloads are valid JSON.
DTOs	APIs use request and response DTOs.
Validation	Required fields, email format, and allowed status values are validated.
Status codes	APIs return 200, 201, 204, 400, and 404 appropriately.
Error messages	Validation messages are clear enough for UI display.
Service layer	Business logic is not written directly in controller actions.
EF Core	Data access is done through ApplicationDbContext.
Async	EF Core calls use async methods where applicable.
Swagger/Postman	All APIs are tested.
README	Setup steps, API list, samples, limitations, and next phase scope are documented.

Official Reference Links

The following official references are also linked inside the task tables above.

- [C# Guide](#)
- [C# Classes and objects](#)
- [C# Collections](#)
- [C# LINQ](#)
- [ASP.NET Core Overview](#)

- [Create controller-based Web API](#)
- [Create Web APIs with ASP.NET Core](#)
- [Routing to controller actions](#)
- [Model binding](#)
- [Model validation](#)
- [Data annotations](#)
- [Dependency injection in ASP.NET Core](#)
- [Controller dependency injection](#)
- [OpenAPI in ASP.NET Core](#)
- [Swagger / OpenAPI help pages](#)
- [EF Core overview](#)
- [EF Core DbContext](#)
- [EF Core InMemory provider](#)
- [EF Core async queries](#)
- [W3C WCAG Error Identification](#)
- [OWASP API Security Top 10](#)